Übungsaufgaben: FPT und Kernelisierung

Algorithmen und Komplexität

Hinweis. Falls nicht anders angegeben: alle Graphen sind ungerichtet. Die Eingabegröße wird mit n bezeichnet. Bei Vertex Cover ist der Parameter k die höchstens erlaubte Anzahl von Cover-Knoten.

Aufgabe 1: FPT oder nur XP?

Ein Algorithmus für ein parametrisiertes Problem hat Eingabegröße $n \geq 2$ und Parameter $k \geq 0$. Untersuchen Sie die folgenden Laufzeitschranken:

$$\begin{aligned} T_1(n,k) &= 2^{k^2} n^3, & T_2(n,k) &= n^{k+2}, \\ T_3(n,k) &= 2^{k \log_2(k+2)} n^2, & T_4(n,k) &= n^{\sqrt{k}+1}, \\ T_5(n,k) &= 2^k n^{10} + n^4. \end{aligned}$$

(a) Welche dieser *angegebenen Laufzeitschranken* haben FPT-Form $f(k)n^{O(1)}$, welche nur XP-Form $n^{g(k)}$? Begründen Sie Ihre Zuordnung.

(b) Klaus sagt: „Ein Algorithmus mit Laufzeit n^{100} ist doch viel langsamer als einer mit Laufzeit $2^k n^2$. Also kann der erste Algorithmus nicht FPT sein."

Hat Klaus recht?

(c) Klaus behauptet zudem: „Der beste Algorithmus, den wir bisher für ein Problem kennen, hat Laufzeit N^k. Also ist das Problem nicht FPT." Hat er recht?

Aufgabe 2: Kernels oder nicht Kernels?

Wir betrachten ein parametrisiertes Problem (D, p). Dabei bezeichnet $k = p(I)$ den Parameter einer Instanz I.

Ein Kernelisierungsalgorithmus ersetzt eine Instanz I in Polynomialzeit durch eine äquivalente Instanz I', sodass

$$|I'| \leq g(k)$$

für eine berechenbare Funktion g gilt.

(a) Muss $|I'| \leq |I|$ gelten? Muss $p(I') \leq p(I)$ gelten?

Begründen Sie jeweils Ihre Antwort anhand der Definition.

(b) Angenommen, die Kernelschranke lautet $g(k) = 100k^2$.

Für eine Instanz I mit $p(I) = 10$ und $|I| = 40$ erzeugt ein Kernelisierungsalgorithmus eine äquivalente Instanz I' mit $|I'| = 600$.

Ist dies mit der Definition eines Kernels vereinbar?

(c) Sei K ein Kernelisierungsalgorithmus mit Kernelschranke $g(k)$.

Konstruieren Sie daraus einen Kernelisierungsalgorithmus K', der zusätzlich garantiert, dass die ausgegebene Instanz nicht größer als die Eingabe ist, d. h.

$$|I'| \leq |I|.$$

Zeigen Sie, dass K' weiterhin ein Kernelisierungsalgorithmus mit Kernelschranke $g(k)$ ist.

Folgern Sie daraus, dass wir ohne Beschränkung der Allgemeinheit annehmen können, dass ein Kernelisierungsalgorithmus die Instanzgröße niemals erhöht.

Aufgabe 3: Der Buss-Kernel in Aktion

Sei $G = (V, E)$ der ungerichtete Graph mit $V = \{a, b, c, x_1, x_2, x_3, y_1, y_2, z_1, z_2\}$ und $E = \{ab, bc, ca, ax_1, ax_2, ax_3, by_1, by_2, cz_1, cz_2\}$.

Wir suchen ein Vertex Cover der Größe höchstens k.

(a) Wenden Sie für $k = 3$ die beiden Reduktionsregeln aus der Vorlesung wiederholt an:

- Isolierte Knoten löschen.
- Falls $\deg(v) > k$, den Knoten v wählen, v und seine inzidenten Kanten löschen und $k \leftarrow k - 1$ setzen.

Geben Sie nach jedem Schritt die verbleibenden Knoten und Kanten sowie den aktuellen Parameter an.

Ist die ursprüngliche Instanz eine Ja-Instanz? Geben Sie gegebenenfalls ein Vertex Cover an.

(b) Wiederholen Sie die Reduktion für $k = 2$. Was folgt für die ursprüngliche Instanz?

(c) Begründen Sie die Korrektheit der Regel $\deg(v) > k$.

Warum wäre die Regel $\deg(v) \geq k$ falsch? Geben Sie ein möglichst kleines Gegenbeispiel an.

Aufgabe 4: Nemhauser–Trotter in Aktion

Sei $G = (V, E)$ der Graph mit

$$V = \{a, b, c, x, y\}$$

und

$$E = \{ab, bc, ca, ax, ay\}.$$

Wir betrachten das Vertex-Cover-Problem mit Parameter k.

(a) Konstruieren Sie den bipartiten Hilfsgraphen B aus der Vorlesung.

Zeigen Sie, dass

$$C_B = \{a, a', b, c\}$$

ein Vertex Cover minimaler Größe von B ist.

(b) Bestimmen Sie die zugehörigen Mengen C_0, V_0 und I_0 aus dem Nemhauser–Trotter-Satz.

(c) Welchen Kernel liefert der Nemhauser–Trotter-Algorithmus für (G, k)?

Entscheiden Sie damit, ob $(G, 2)$ und $(G, 3)$ Ja-Instanzen sind.

Aufgabe 5: Ein Kernel für Subset Sum

Beim Problem Subset Sum sind positive ganze Zahlen $a_1, \ldots, a_n$ und eine positive ganze Zahl t gegeben. Gesucht ist eine Teilmenge der *Positionen* $I \subseteq \{1, \ldots, n\}$ mit

$$\sum_{i \in I} a_i = t.$$

Dabei dürfen Zahlen mehrfach vorkommen. Beispielsweise können bei der Eingabe $2, 2, 3$ beide Zweien unabhängig voneinander ausgewählt werden.

Wir betrachten Subset Sum mit dem *Parameter* t.

(a) Zeigen Sie, dass die folgenden beiden Reduktionsregeln die Antwort auf die Eingabe nicht verändern:

  (i) Entferne alle Zahlen $a_i > t$.

  (ii) Falls eine Zahl $a \leq t$ häufiger als $\lfloor t/a \rfloor$-mal vorkommt, entferne die überzähligen Kopien.

(b) Zeigen Sie, dass nach Anwendung dieser beiden Regeln höchstens t^2 Zahlen übrig bleiben.

(c) Beschreiben Sie einen Kernelisierungsalgorithmus für Subset Sum mit Parameter t.

  Begründen Sie seine Korrektheit und zeigen Sie, dass er in Polynomialzeit arbeitet und einen Kernel polynomialer Größe in t liefert.

(d) Erklären Sie, wie sich aus diesem Kernel ein FPT-Algorithmus für Subset Sum gewinnen lässt.

  Eine einfache Laufzeitschranke genügt; der Algorithmus muss nicht besonders effizient sein.

(e) Betrachten Sie die Instanz

$$A = (2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 5, 5, 5, 5, 5, 17, 21), \qquad t = 10.$$

  (i) Wenden Sie die beiden Reduktionsregeln an und geben Sie die verbleibende Liste an.

  (ii) Entscheiden Sie, ob die reduzierte Instanz eine Ja-Instanz ist.

  (iii) Wie viele Teilmengen der Positionen gibt es in der reduzierten Instanz? Wie viele davon müsste der Algorithmus aus (d) höchstens untersuchen?